Embedded Systems Project 1

Conor King C22303056 20/03/2026

OLLSCOIL TEICNEOLAÍOCHTA
BHAILE ÁTHA CLIATH

TU DUBLIN

TECHNOLOGICAL
UNIVERSITY DUBLIN

Contents

Introduction 4
Elements of Components 4
1. STM32L432KC 4
2. Serial Display (OLED Display (0.96" SPI) 4
3. Serial Monitor 4
4. Accelerometer 5
5. LED 5
6. Timers 6
7. Button 6
8. Wires/Cables 6
Schematic 6
Development Process 9
Step 1: Initial Design Idea 9
Step 2: Initial Implementation and Issue Encountered 9
Step 3: Isolating the Problem 9
Step 4: Temporary Design Change 9
Step 5: Hardware Issue Identified 9
Step 6: Debugging Using Serial Monitor 10
Step 7: Developing Tilt Detection 10
Step 8: Implementing LED Feedback 10
Step 9: Building the Password System 10
Step 10: Final Testing and Refinement 10
Code Section 11
Tilt Detection 11
Password Logic 11
Timer Usage 13
Accelerator (I2C) 13
Future Improvements 14
Reference 14

Figure 1 - Accelerometer axis 5
Figure 2 - DESIGN SCHEMATIC OF PASSWORD SYSTEM WITH CORRECT WIRING..... 7
Figure 3 - Breadboard wiring OVERVIEW PHOTO 8
Figure 4 - Tilt Detection Code 11
Figure 5 - Main Tilt Password setup 12
Figure 6 - Tilt detection progress logic 13
Figure 7 - Accelerometer logic sending/ receiving 14

Introduction

In this design specification the full project will be discussed in detail, both the decisions and tasks undertaken to complete and ultimately have a working system that utilises all most if not all elements undertaken within the first few weeks during this module. The board used is the STM32L432KC. This project was chosen as tilt sensing is a practical real-world application of accelerometers used in devices such as smartphones and tablets [1]. ADC and DAC peripherals were not used in this design as all inputs and outputs were digital, using I2C, SPI and GPIO communication, therefore no analog signal conversion was required.

Such elements include I2C protocol, Spi protocol, GPIO, timers, debugging techniques, uart and much more.

Elements of Components

1. STM32L432KC

➔ The main brain of this assignment is the STM32L432KC and operates the code, reads sensor data form the accelerometer, controls led and checks password logic. The STM32 Board basically has all elements of the project included because practically all elements are controlled or flow through it.

2. Serial Display (OLED Display (0.96" SPI)

➔ The serial display is used to visually show digitalised data. In this case how long it took a user to complete one instance of the password sequence. It also displays the words “locked” or “unlocked” in green depending on whether the device is as stated. The serial device uses Spi protocol (view below schematic for any information on connections). Spi (Serial Peripheral Interface) is a mostly one way sending of data from a master component, STM32L432KC, to a slave element, the oled display. The main line that’s sends the data is MOSI, MASTER OUT, SLAVE IN.The clock line keeps everything in sink and tells the display when to read the data. Every one pulse sends one bit for example. The CS line turns high when receiving data and low when it is not. Tells the OLED what data is being sent.

3. Serial Monitor

➔ Serial monitor is great for real time data collection when the pc is connected to the board. It is especially prolific with the accelerometer. In this case the serial monitor is used for debugging and there will be more on that in the sections that follow. The accelerometer is monitored here as well as the switches status, on or off. It also signals in words when a left, right, forward or back action is taken.

4. Accelerometer

➔ The accelerometer is used in terms of its x and y axis in this case for simplicity completely ignoring the z-axis. Theoretically using the z-axis would grant more movements and possible password actions. The password can be manually changed in the code but remain, LEFT, RIGHT, FORWARD and BACKWARDS. The accelerometer uses ICT protocol to communication. The SCL is the line that controls timing via a clock line. The SDA line both sends and receives data over a single line that uses pull up resistors. The accelerometer is told what to do by the board and then sends data to the board to receive. This is common two-way communication over a single bus.

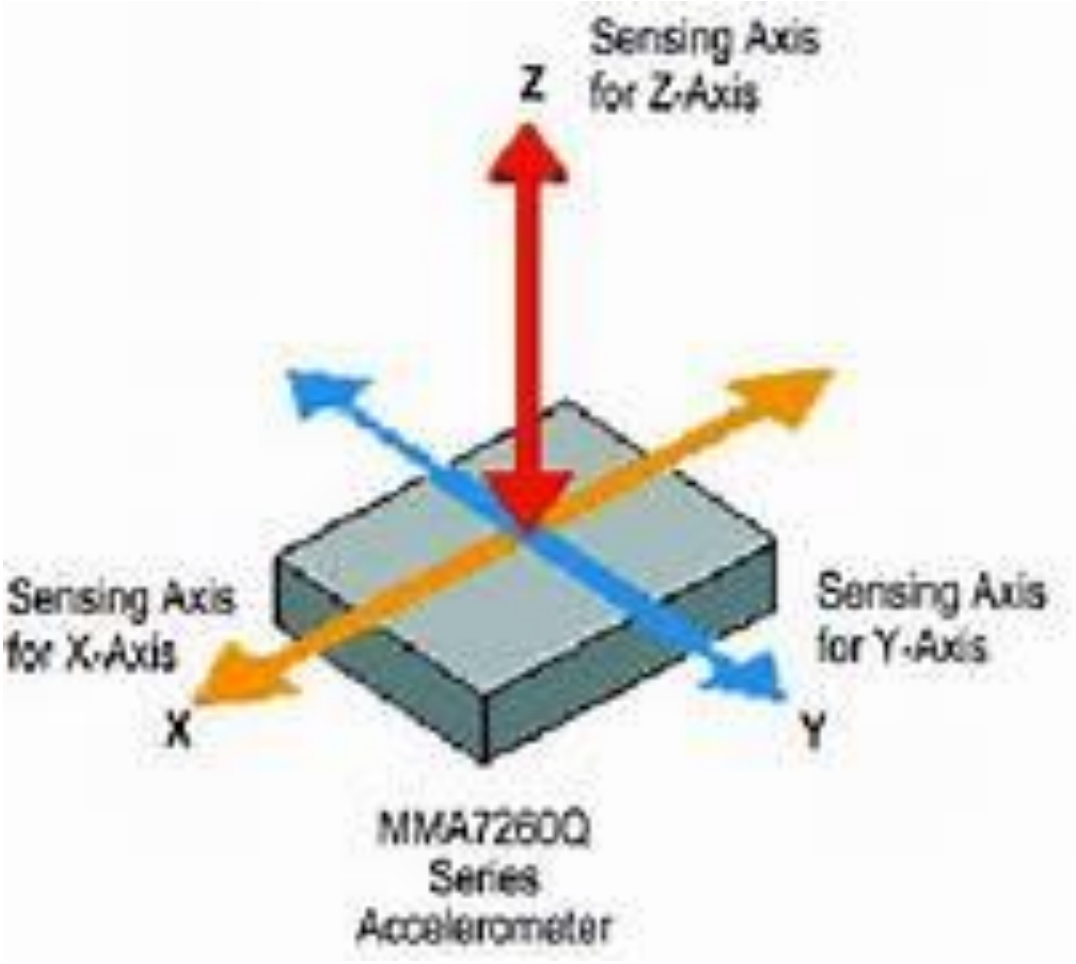
➔ 

Figure 1 - Accelerometer axis

5. LED

➔ The LED's visually show correct password attempts and are set up in a way that uses a single PMW port to activate two LEDs separately, red to green. The same thing is done to visually show the system is running with larger LEDs. Once that button is pressed it activates a few different timers.

6. Timers

- ➔ In this system, timing is handled using the SysTick timer, which updates a variable called milliseconds every 1 ms. This timer is used to measure how long it takes to unlock the system, reset the password if too much time (5 seconds) passes between inputs, and create small delays for things like button pressing and slowing down the loop. Overall, the timer helps keep track of time and ensures the system runs smoothly and reliably.
- ➔ Theres also a function to count and output how long a password sequence took to complete by counting how long the system has been on for and how long from button press it the completion time was.

7. Button

- ➔ The button is a simple push signal button. It sends the line neutral when pressed so its always live by default.

8. Wires/Cables

- ➔ Wires and cables are regular tinned copper wire with plastic insulation. The colours for the most part are consistent. Red for live 3.3 volts and black for ground. Other colours are a bit more random.

Schematic

The schematic shown below is a further representation of the other below figure of the actual system for ease of understanding. The physical breadboard was designed to be as neat as possible with a nice consistent set up.

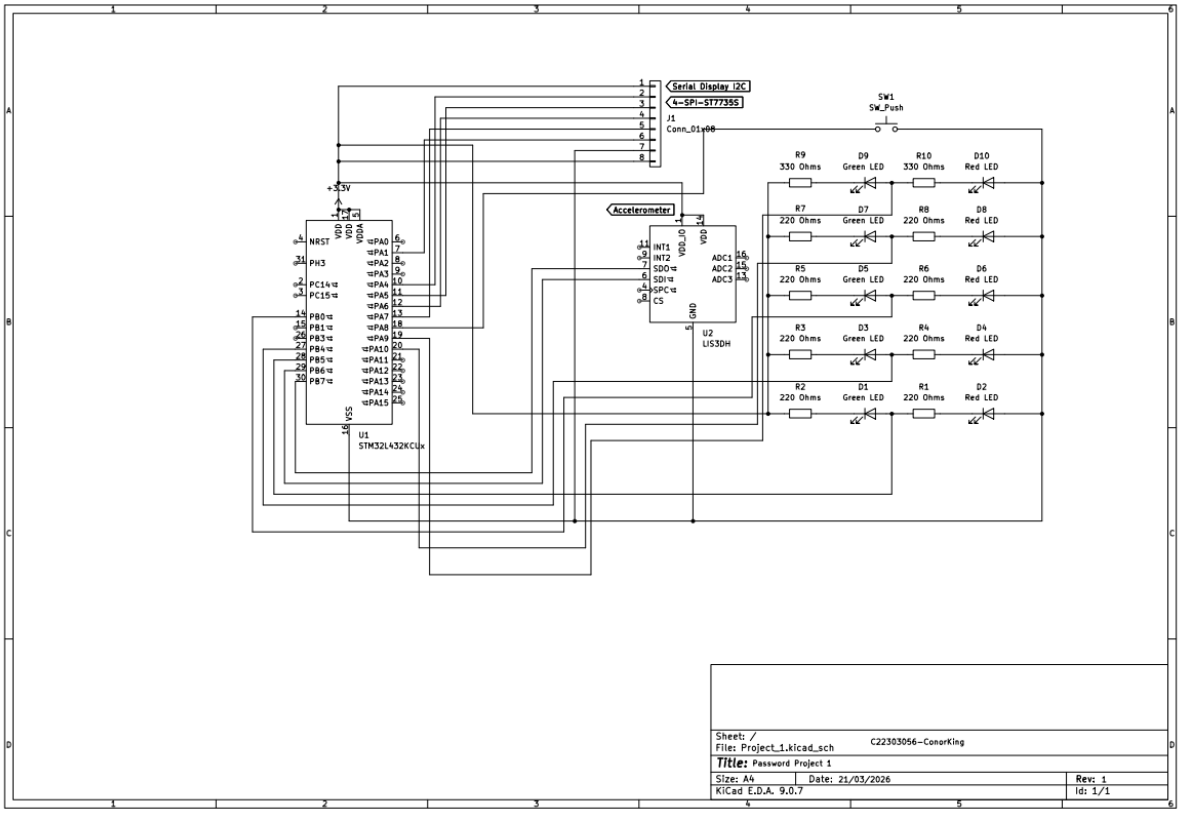


Figure 2 - DESIGN SCHEMATIC OF PASSWORD SYSTEM WITH CORRECT WIRING

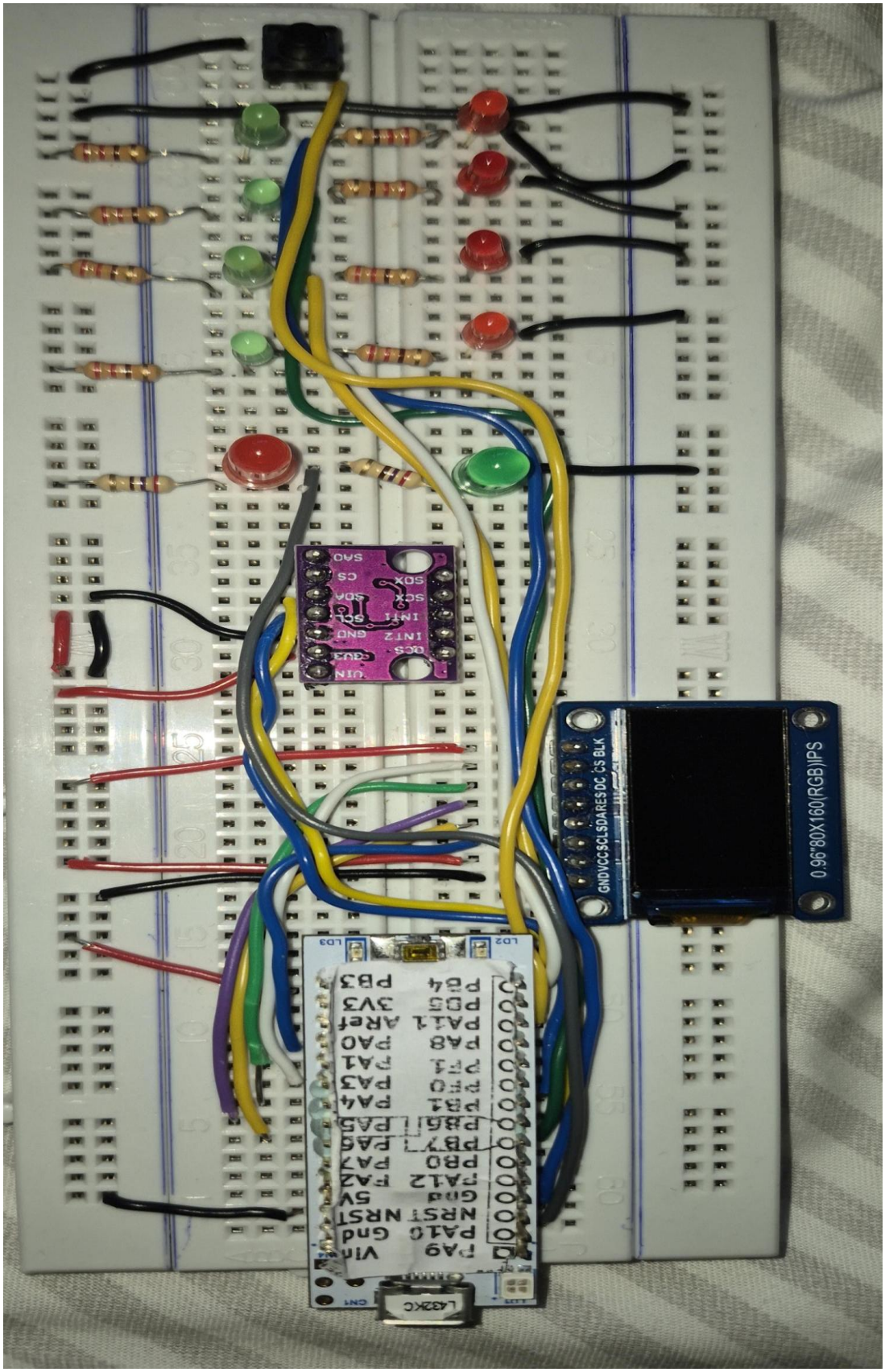


Figure 3 - Breadboard wiring OVERVIEW PHOTO

Development Process

The project was developed in a bit goal by goal method, with testing and troubleshooting carried out at each different stages to obtain smaller goals to achieve the main overall goal.

Step 1: Initial Design Idea

The original design was to use the OLED display as the main output, where the screen would indicate when the correct password had been entered. The LEDs were not intended to be used, as the display was meant to provide all user feedback.

Step 2: Initial Implementation and Issue Encountered

During early testing, the OLED display did not work as expected. This prevented further progress, as the main output of the system was not functioning.

Step 3: Isolating the Problem

To identify the issue, each component was tested individually:

- The accelerometer was tested first using the serial monitor to verify that valid X and Y values were being received. This confirmed that the I2C communication was working correctly and that the sensor was functioning as expected.
- The OLED display was then tested separately, but no output was observed. This helped confirm that the issue was related to the display or its hardware connections rather than the code.

This indicated that the issue was specific to the display or its connections.

Step 4: Temporary Design Change

Due to the display issue and time constraints, the OLED was removed from the system. The design was modified to use LEDs as indicators instead, allowing development to continue without relying on the display.

Step 5: Hardware Issue Identified

With assistance from the lecturer, it was later discovered that the issue with the display was caused by the hardware connections. Certain pins were linked in a way that interfered with proper operation. A capacitor connected underneath the board was removed, which allowed the pins to operate independently. This explained why the display had not been working earlier.

Step 6: Debugging Using Serial Monitor

Throughout development, the serial monitor was used as the main debugging tool. It allowed real-time monitoring of accelerometer values and system behaviour. This made it easier to verify that tilt detection was working correctly and that the system was responding to inputs as expected.

- X and Y values were being read correctly
- Tilt detection logic was functioning

Step 7: Developing Tilt Detection

The first tilt tested was left tilt. Threshold values for the X and Y axes were adjusted experimentally until the system could reliably detect this movement. This testing ensured that only deliberate movements triggered a response and helped eliminate false detections.

- right
- forward
- backward

Step 8: Implementing LED Feedback

After tilt detection was working, LEDs were introduced to provide visual feedback:

- Each tilt direction was tested to ensure the correct LED responded, verifying that the GPIO outputs were functioning correctly. Once confirmed, the LEDs were used to indicate password progress, with each correct input activating the next LED.
- This allowed progress through the password to be clearly shown

Step 9: Building the Password System

The individual tilt detections were combined into a sequence-based password. The system was tested by inputting the full sequence and observing whether:

- progress increased correctly
- incorrect inputs were ignored
- the system reset when needed

Step 10: Final Testing and Refinement

The complete system was tested multiple times to verify overall operation.

- threshold values

- timing delays
- overall responsiveness

This resulted in a working tilt-based password system with reliable detection and clear user feedback.

Code Section

This section will focus on main functions and elements within the code.

Tilt Detection

This small snippet of code shows an example of the tilt detection function where raw accelerometer data is turned into a left, right back or forward function based on real time movement. The logic is only activated based on if the movements are gone past a separate threshold, therefore supressing possibilities of accidentally activating the function for each element. If there is no tilt then it returns nothing. In saying the actual hard thresholds are listed above where everything is configured and initilised.

```
static uint8_t detect_tilt(int32_t x_g100, int32_t y_g100)
{
    if ((y_g100 < 200) && (y_g100 > -200))
    {
        if (x_g100 > X_THRESHOLD) return TILT_RIGHT;
        if (x_g100 < -X_THRESHOLD) return TILT_LEFT;
    }

    if ((x_g100 < 300) && (x_g100 > -300))
    {
        if (y_g100 > Y_FORWARD_THRESHOLD) return TILT_FORWARD;
        if (y_g100 < Y_BACK_THRESHOLD) return TILT_BACK;
    }

    return TILT_NONE;
}
```

Figure 4 - Tilt Detection Code

Password Logic

This is the basic logic that allows the password to be changed. As previously mentioned, the password can be manually configured however is required.

```
static const uint8_t password[PASSWORD_LENGTH] =
{
    TILT_LEFT,
    TILT_RIGHT,
    TILT_FORWARD,
    TILT_BACK
};
```

Figure 5 - Main Tilt Password setup

This section of code controls how tilt inputs are accepted and checked against the password sequence. The condition ready && (tilt! = TILT_NONE) ensures that a new input is only processed when the system is ready and a valid tilt has been detected. The ready flag prevents the same movement from being detected multiple times.

Once a valid tilt is detected, the system sets ready = 0 to block further inputs until the device returns to a neutral position. The detected tilt is then compared with the expected value in the password sequence using password[progress]. If the input is correct, the progress counter is incremented, the input time is recorded for timeout tracking, and the LED display is updated to reflect the current progress.

```
if (ready && (tilt != TILT_NONE))
{
    ready = 0;

    if (tilt == password[progress])
    {
        progress++;
        last_correct_input_time = milliseconds;
        show_progress(progress);
    }
}
```

Figure 6 - Tilt detection progress logic

Timer Usage

The SysTick timer is used to track time and implement a timeout feature. If the user takes too long the password sequence will reset.

```
if ((progress > 0) && ((milliseconds - last_correct_input_time) >= PASSWORD_TIMEOUT_MS))
{
    progress = 0;
    leds_red();
}
```

Accelerator (I2C)

This function reads digital data from the accelerometer using I2C communication by accessing specific register addresses.

```c
static int16_t accel_read_axis(uint8_t reg)
{
    uint16_t low, high;

    I2CStart(ACCEL_ADDR, WRITE, 1);
    I2CWrite(reg);
    I2CReStart(ACCEL_ADDR, READ, 2);

    low = I2CRead();
    high = I2CRead();

    I2CStop();

    return (int16_t)(low | (high << 8));
}
```

Figure 7 - Accelerometer logic sending/ receiving

Future Improvements

One improvement would be the inclusion of ADC or DAC functionality. These were not used in this project as all inputs and outputs were digital; however, they could be added to support analog sensors or output signals such as audio feedback. The OLED display could also be fully integrated for improved user feedback. Further improvements include allowing dynamic password changes, using the Z-axis for more complex inputs, and adding security features such as lockout after multiple failed attempts.

Reference

[1] M. Pedley, *Tilt Sensing Using a Three-Axis Accelerometer*, Freescale Semiconductor, Application Note AN3461, Rev. 6, Mar. 2013.